

Unit - 10 : Regular Expressions

1. Introduction

- Regular Expression (Regex) एक pattern (पैटर्न) होता है जो text (string) में specific pattern को search, match, validate या manipulate करने के लिए उपयोग किया जाता है।
- यह किसी भी text data में characters के sequence को define करने का एक powerful तरीका है।
- Regular Expression का उपयोग text processing, data validation, search, replace और information extraction जैसे tasks के लिए किया जाता है।
- Python में re module regular expression operations के लिए provide किया जाता है।
- Regex का pattern string के रूप में लिखा जाता है और उसे re module की functions के साथ use किया जाता है।

"Regular Expression एक छोटा सा pattern है लेकिन powerful tool है जो text data को handle करने के लिए बहुत useful है।"

2. Why use Regular Expressions?

- Complex text search को आसान बनाता है।
- Data validation (जैसे email, phone number, date) के लिए useful है।
- Text से specific information extract करने में मदद करता है।
- Search और replace operations को fast और flexible बनाता है।
- Programming और real world applications दोनों में widely used होता है।

3. re Module in Python

- Python में regular expression के लिए re (regular expression) module use किया जाता है।
- यह module pattern matching के लिए कई useful functions provide करता है।
- सबसे commonly used functions :

Function	Description
re.match()	String के start में match खोजता है
re.search()	पूरी string में कहीं भी match खोजता है
re.findall()	सभी matches की list return करता है
re.sub()	Match हुए text को replace करता है
re.split()	Pattern के आधार पर string को split करता है

Note :

re module को use करने से पहले import करना होता है।
import re

4. Regular Expression Pattern

- Regular expression pattern में normal characters के साथ कुछ special characters (metacharacters) भी होते हैं।
- ये special characters pattern को define करने के लिए use किए जाते हैं।

Example :

Pattern : ^a.*b\$

^ → string की शुरुआत को दर्शाता है

a → 'a' character से start

.* → उसके बाद कोई भी characters (0 या अधिक)

b → 'b' character

\$ → string के अंत को दर्शाता है

5. Simple Character Matches

- Simple character match में हम सीधे characters का use करके pattern बनाते हैं।
- यह basic matching के लिए use होता है जहाँ हम specific characters को search करना चाहते हैं।
- उदाहरण के लिए, किसी string में कोई particular letter, digit या word को ढूँढना।

6. Common Simple Character Examples

Pattern	Meaning	Example String	Match Result
a	character 'a'	apple	a (match)
abc	exact string 'abc'	abcdef	abc (match)
Python	exact word 'Python'	I love Python	Python (match)
7	digit '7'	a7b	7 (match)
xyz	exact string 'xyz'	xyz123	xyz (match)

7. Example Program

```
import re
text = "I love Python and Python is easy."
# search() example
match = re.search("Python", text)
if match:
    print("Found:", match.group())
else:
    print("Not Found")

# findall() example
matches = re.findall("Python", text)
print("All matches:", matches)
```

Output :

Found: Python
All matches: ['Python', 'Python']

8. Key Points

- Regular expressions text को search और manipulate करने के लिए एक powerful tool हैं।
- Simple character matches basic pattern matching के लिए use होते हैं।
- Python में re module regular expression operations के लिए provide किया जाता है।
- Regular expressions का practice करने से complex patterns को समझना आसान हो जाता है।

Topic : Special Characters , Character Classes

1. Special Characters (Metacharacters)

- Regular Expressions में कुछ characters ऐसे होते हैं जिनका special meaning होता है, इन्हें Special Characters या Metacharacters कहते हैं।
- यह characters normal text की तरह match नहीं होते, बल्कि pattern को define करने के लिए use होते हैं।
- इनका उपयोग search, validate और manipulate करने के लिए किया जाता है।

Special Character	Meaning	Example / Use
.	किसी भी single character	a.t → act, a1t
^	string की शुरुआत	^a → apple
\$	string का अंत	t\$ → cat, bat
*	0 या अधिक बार	ab* → a, ab, abb
+	1 या अधिक बार	ab+ → ab, abb
?	0 या 1 बार (optional)	colou?r → color, colour
	either / or (alternative)	cat dog → cat या dog
()	grouping	(ab)+ → ab, abab
[]	character set	[abc] → a, b या c
{ }	exact repetition	a{3} → aaa
\	escape character	\. → dot को match
\d	digit (0-9)	\d → 0 to 9
\w	word character	\w → a-z, A-Z, 0-9, _
\s	whitespace	\s → space, tab, newline

2. Character Classes (Custom Set)

- Square brackets [] का use करके हम अपना खुद का character set define कर सकते हैं।
- इसमें दिए गए किसी भी character में से एक character match होता है।
- Range का भी use कर सकते हैं, जैसे a-z, A-Z, 0-9.
- यदि ^ को brackets के अंदर लिखा जाए तो वह negation (उसके अलावा) को show करता है।

Pattern	Meaning	Example Match
[abc]	a, b या c में से कोई एक	a, b, c
[a-z]	छोटे अंग्रेजी अक्षर (a से z)	a, m, z
[A-Z]	बड़े अंग्रेजी अक्षर (A से Z)	A, M, Z
[0-9]	digits (0 से 9)	0, 5, 9
[a-zA-Z]	सभी letters	a, Z, m
[0-9a-z]	digit या lowercase letter	5, a, 9
[^abc]	a, b, c को छोड़कर कोई भी	d, x, 1
[^0-9]	digit के अलावा कोई भी	a, #, @

3. Character Classes (Predefined)

- Character Classes कुछ pre-defined sets होते हैं जो characters के group को represent करते हैं।
- इनका use frequently होता है क्योंकि ये commonly used patterns के लिए shortcut प्रदान करते हैं।

Character Class	Meaning	Equivalent
\d	Digit (0-9)	[0-9]
\D	Non-digit	[^0-9]
\w	Word character	[a-zA-Z0-9_]
\W	Non-word character	[^\w]
\s	Whitespace	[\t\n\r\f\v]
\S	Non-whitespace	[^\s]

Note :

Character classes की मदद से हम बड़े patterns को छोटे और readable form में लिख सकते हैं।

4. Examples of Character Classes

Pattern	Matches	Example Strings
\d+	one or more digits	123, 007
\w+	word (letters/digits/_)	name_1, test
\s+	one or more spaces	. (space) , \t
\D+	non-digit characters	abc, #\$
\W+	non-word characters	@#!, .
\S+	non-space characters	abc, 123
[a-z]+	lowercase letters	abc, hello
[A-Z]+	uppercase letters	ABC, PYTHON
[0-9]{3}	exact 3 digits	123, 456
[a-zA-Z0-9_]+	alphanumeric word	user_1, test22

5. Key Points

- Special characters pattern को powerful बनाते हैं।
- Character classes से हम group of characters को easily match कर सकते हैं।
- Predefined classes (\d, \w, \s आदि) commonly use होते हैं।
- Custom character sets [] से हम अपनी आवश्यकता के अनुसार pattern बना सकते हैं।
- Regular expressions का सही उपयोग text processing, validation और data extraction के लिए बहुत useful है।

Topic : Quantifiers, The Dot Character, Greedy Matches

1. Quantifiers (मात्रा सूचक)

- Quantifiers regular expressions में बताते हैं कि किसी character या group की कितनी बार occurrence होनी चाहिए।
- इनका use pattern में repetition specify करने के लिए होता है।
- ये character, group या character class पर apply होते हैं।
- Quantifiers की मदद से हम flexible और powerful patterns बना सकते हैं।

Quantifier	Meaning	Example	Matches
*	0 या अधिक बार	a*	'' , a, aa, aaa
+	1 या अधिक बार	a+	a, aa, aaa
?	0 या 1 बार (optional)	colou?r	color, colour
{n}	ठीक n बार	a{3}	aaa
{n,}	कम से कम n बार	a{2,}	aa, aaa, aaaa
{n,m}	n से m बार	a{2,4}	aa, aaa, aaaa

Example :

Pattern : a+	String : "aabb"	Match : "aaa"
Pattern : ab*	String : "abbb"	Match : "abbb"
Pattern : colou?r	String : "colour"	Match : "colour"
Pattern : \d{2,4}	String : "1234"	Match : "1234"

Key Points :

- * → 0 या अधिक बार
- + → 1 या अधिक बार
- ? → 0 या 1 बार
- {n}, {n,}, {n,m} → specific range के लिए
- Quantifiers greedy by default होते हैं (ज्यादा से ज्यादा match करते हैं)।

2. The Dot Character (.)

- Dot (.) एक special character है जो किसी भी single character को match करता है (उसके type की परवाह किये बिना)।
- यह letter, digit, symbol, space आदि सभी को match करता है।
- लेकिन यह newline character (\n) को match नहीं करता (by default)।
- Dot character का use तब करते हैं जब हमें किसी भी एक character की जगह match करना हो।

Note : अगर newline को भी match करना हो तो re.DOTALL flag का use किया जाता है।

Pattern	String	Match	Explanation
a.b	'acb'	acb	बीच का कोई भी character
a.b	'a b'	a b	space भी match होता है
a.b	'a#b'	a#b	symbol भी match होता है
a.b	'ab'	No match	बीच में character नहीं है
a.*b	'axxb'	axxb	.* के साथ 0 या अधिक character match होते हैं।

Common Use :

- file name matching → *.txt
- email pattern → a.b@gmail.com
- any character search → p.t (pat, pot, put...)

3. Greedy Matches (लालची मिलान)

- Python में quantifiers (*, +, ?, {n,m}) by default greedy होते हैं, जिसका मतलब वे सबसे लंबा (longest) possible match करते हैं।
- Greedy matching useful होता है, लेकिन कुछ cases में यह unwanted results दे सकता है।
- ऐसे situations में non-greedy (lazy) quantifiers का use किया जाता है, जिसमें quantifier के बाद ? लगाया जाता है (*?, +?, ??, {n,m}??)।

Pattern	String	Greedy Match	Non-greedy Match
<.*>	'<html>'	<html>	<html> (same)
<.*>	'<a> '	<a> 	<a> (with .*?)
a.*b	a123b456b'	a123b456b	a123b (with .*?)
".*"	"hello" "world"	"hello" "world"	"hello" (with .*?)

Key Points :

- Greedy match → सबसे लंबा match करता है।
- Non-greedy match → सबसे छोटा match करता है (quantifier के बाद ?)।
- Use non-greedy matching जब unwanted extra text भी match हो रहा हो।

Topic : Grouping , Matching at Beginning or End , Match Objects , Substituting

1. Grouping ()

- Parentheses () का use करके हम pattern के किसी part को group कर सकते हैं।
- Grouping से हम complex pattern को easily handle कर सकते हैं और उस group पर quantifiers apply कर सकते हैं।
- यह matching को organize करने के लिए useful होता है।
- Grouping की help से हम sub-patterns को capture भी कर सकते हैं (जिन्हें groups कहा जाता है)।

Pattern	Meaning	Example String	Match
(ab)+	'ab' group एक या अधिक बार	ab, abab	ab, abab
(a b)c	'a' या 'b' के बाद 'c'	ac, bc	ac, bc
(cat dog)	'cat' या 'dog'	I have a dog	dog
(\d{3})-(\d{2})	3 digits - 2 digits	123-45	123-45

Example :

```
import re
text = "cat and dog and cat"
pattern = r"(cat|dog)"
matches = re.findall(pattern, text)
print(matches)      # Output: ['cat', 'dog', 'cat']
```

3. Match Objects

- re module की functions (जैसे re.search(), re.match()) हमें एक Match Object return करती हैं।
- Match Object में matched text और उसके related information प्राप्त करने के लिए कई methods होती हैं।
- Match object का use तब किया जाता है जब हमें सिर्फ first match की information चाहिए।
- यदि match नहीं मिलता तो None return होता है।

Method	Description
group()	पूरा matched text return करता है।
start()	match की starting index देता है।
end()	match की ending index देता है।
span()	(start, end) tuple return करता है।
group(n)	किसी specific group का matched text देता है।

Example :

```
import re
text = "My roll number is 1023."
m = re.search(r"\d+", text)
if m:
    print("Matched text :", m.group())      # 1023
    print("Start index :", m.start())      # 19
    print("End index :", m.end())          # 23
    print("Span :", m.span())              # (19, 23)
```

2. Matching at Beginning or End

- ^ (caret) का use string के beginning में match करने के लिए होता है।
- \$ (dollar) का use string के end में match करने के लिए होता है।
- ये anchors पूरे string में position को check करते हैं।
- इनका use data validation (जैसे पूरा string match करना) के लिए किया जाता है।

Pattern	Meaning	Example String	Match
^abc	string की शुरुआत में 'abc'	abcdef	abc
xyz\$	string के अंत में 'xyz'	helloxyz	xyz
^\d+\$	पूरी string केवल digits	12345	12345
^[A-Z]	string की शुरुआत में uppercase letter	Python	P

Example :

```
import re
text = "hello world"
print(re.findall(r"^hello", text))      # ['hello']
print(re.findall(r"world$", text))      # ['world']
```

4. Substituting

- re module में re.sub() function का use matched text को किसी दूसरे text से replace करने के लिए होता है।
- यह function सभी matches या specified number (count) तक के matches को replace कर सकता है।
- यह text को modify करने के लिए बहुत useful है, जैसे extra spaces हटाना, format बदलना, आदि।

Syntax :

```
re.sub(pattern, replacement, string, count=0)
```

↓ जिसे search करना है
 ↓ जिससे replace करना है
 ↓ जिस text में search करना है
 ↓ कितनी बार replace करना है (0 = सभी)

Example :

```
import re
text = "cat cat dog cat"
new_text = re.sub(r"cat", "pet", text)
print(new_text)      # pet pet dog pet

# सिर्फ first match replace करना
new_text2 = re.sub(r"cat", "pet", text, count=1)
print(new_text2)      # pet cat dog cat
```


Topic : Splitting a String , Compiling Regular Expressions , Flags

1. Splitting a String

- re module में re.split() function का use किसी string को split करने के लिए किया जाता है।
- यह specified pattern के आधार पर string को parts में divide करता है।
- Split करते समय delimiter (जैसे space, comma या कोई regex pattern) remove हो जाता है।
- यह function list return करता है जिसमें split हुए substrings होते हैं।
- यदि maxsplit दिया जाए तो केवल उतनी बार ही split किया जाता है।

Syntax :

```
re.split(pattern, string, maxsplit = 0, flags = 0)
```

- pattern - वह regex pattern जिसके आधार पर split करना है।
- string - वह input string जिसे split करना है।
- maxsplit - (optional) अधिकतम कितनी बार split करना है।
- flags - (optional) regex flags (जैसे re.IGNORECASE)।

Example 1 : (Basic Split)

```
import re
text = "apple,banana,mango,orange"
result = re.split(",", text)
print(result) # ['apple', 'banana', 'mango', 'orange']
```

Example 2 : (Split using multiple spaces)

```
import re
text = "This is a test"
result = re.split(r"\s+", text)
print(result) # ['This', 'is', 'a', 'test']
```

Example 3 : (Using maxsplit)

```
import re
text = "one-two-three-four"
result = re.split(r "-", text, maxsplit = 2)
print(result) # ['one', 'two', 'three-four']
```

Note : re.split() में delimiter result में include नहीं होता है, जबकि re.findall() match को return करता है।

Example : (IGNORECASE)

```
import re
text = "Python is Powerful"
pattern = re.compile(r"python", re.IGNORECASE)
match = pattern.search(text)
print(match.group()) # Python
```

Key Point : Flags regex को powerful और flexible बनाते हैं, जिससे अलग-अलग types के text patterns को आसानी से handle किया जा सकता है।

2. Compiling Regular Expressions

- Python में re.compile() function का use regex pattern को compile करने के लिए किया जाता है।
- Compiled pattern एक Pattern Object return करता है।
- इस Pattern Object का use बार-बार search, match, findall, split आदि operations करने के लिए किया जा सकता है।
- यदि हम एक ही pattern को कई बार use कर रहे हैं तो compile करना ज्यादा efficient होता है।
- यह code को readable भी बनाता है और performance भी improve करता है।

Syntax :

```
pattern = re.compile(pattern, flags = 0)
```

- pattern - regex pattern (string)
- flags - (optional) जैसे re.IGNORECASE, re.MULTILINE

Example :

```
import re
pattern = re.compile(r"\d+") # compile pattern
text = "Roll no: 101, 102, 205"
matches = pattern.findall(text)
print(matches) # ['101', '102', '205']
```

Note : Compiled pattern का use करने के लिए हम उस object के methods (search(), findall(), split()) आदि को call करते हैं।

3. Flags

- Flags का use regex pattern के behavior को modify करने के लिए किया जाता है।
- यह हमें कुछ special options provide करते हैं, जैसे case-insensitive search, multiline mode आदि।
- Flags को re.compile() या सीधे functions (जैसे re.search()) में use किया जा सकता है।

Flag	Name	Description
re.IGNORECASE	I	Case insensitive matching (uppercase/lowercase ignore)
re.MULTILINE	M	^ और \$ हर line पर work करते हैं।
re.DOTALL	S	Dot (.) newline (\n) को भी match करता है।
re.VERBOSE	X	Pattern को readable form में लिखने के लिए (spaces और comments allow)।

Example : (MULTILINE)

```
import re
text = "Hello\nWorld"
pattern = re.compile(r"^World", re.MULTILINE)
match = pattern.search(text)
print(match.group()) # World
```


Program : Validate Email Addresses using Regular Expression

1. Problem Statement

- एक Python program लिखिए जो किसी string में दिए गए email address को validate करे (check करे) कि वह valid email format में है या नहीं, using Regular Expression.

2. Approach

- Email का general format होता है :

username@domain.extension

- हम regular expression pattern का use करके email के format को match करेंगे।
- यदि string pattern से match करता है तो वह valid email है, अन्यथा invalid है।
- इसके लिए हम re module की re.match() function का use करेंगे।

3. Regular Expression Pattern

Pattern : `r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"`

Pattern Explanation :

Part	Meaning
<code>^</code>	string की शुरुआत (start)
<code>[A-Za-z0-9._%+-]+</code>	username part (एक या अधिक valid characters)
<code>@</code>	'@' symbol (mandatory)
<code>[A-Za-z0-9.-]+</code>	domain name (जैसे gmail, yahoo)
<code>\.</code>	dot (.) before extension
<code>[A-Za-z]{2,}</code>	extension (जैसे .com, .in, .org)
<code>\$</code>	string का अंत (end)

4. Program Code (Python)

```
import re
# email validation के लिए regex pattern
pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
def check_email(email):
    if re.match(pattern, email):
        return "Valid Email"
    else:
        return "Invalid Email"
# Test cases
emails = [
    "student@gmail.com",
    "user.name@yahoo.in",
    "abc123@domain.co",
    "wrong@.com",
    "noatsymbol.com",
    "test@domain",
    "name+tag@university.ac.in"
]
for e in emails:
    print(f"{e} --> {check_email(e)}")
```

5. Output

```
student@gmail.com --> Valid Email
user.name@yahoo.in --> Valid Email
abc123@domain.co --> Valid Email
wrong@.com --> Invalid Email
noatsymbol.com --> Invalid Email
test@domain --> Invalid Email
name+tag@university.ac.in --> Valid Email
```

6. Explanation of Output

- पहले, दूसरे, तीसरे और आखिरी email का format regex pattern से match करता है, इसलिए यह Valid Email है।
- wrong@.com में domain name सही नहीं है (dot के बाद domain missing है), इसलिए Invalid है।
- noatsymbol.com में '@' symbol ही नहीं है, इसलिए Invalid है।
- test@domain में extension नहीं है, इसलिए Invalid है।

7. Key Points

- Regular Expression complex pattern matching के लिए बहुत useful होता है।
- Email validation में regex का use करके हम format को quickly check कर सकते हैं।
- re.match() function string की शुरुआत से match करता है।
- Pattern को और strict या flexible बनाया जा सकता है requirement के अनुसार।
- Regex validation 100% perfect नहीं होता, लेकिन basic format checking के लिए काफी अच्छा है।

8. Applications

- User registration form में email validation।
- Data cleaning और preprocessing के लिए।
- Web scraping में email extract करने के लिए।
- Any application जहाँ specific format validation की आवश्यकता हो।

Note :

Regular Expression का use करते समय pattern को समझना बहुत जरूरी है, क्योंकि छोटा सा change भी matching result को प्रभावित कर सकता है।